Title

Author: Name

Supervisor: Name

A thesis submitted in partial fulfilment of the
requirements for the degree of
MSc Mathematical Modelling and Machine Learning

School of Mathematical Sciences,
University College Cork,
Ireland

September 2025

Declaration of Authorship

This report is wholly the work of the author, except where explicitly stated otherwise. The source of any material which was not created by the author has been clearly cited.

Date: 11/08/2026

Signature:

Acknowledgements

I would like to thank...

Abstract

A brief overview of the thesis...

Contents

List of Figures

Chapter 1

Theoretical Background

The Anser electromagnetic tracking (EMT) system infers the position and orientation (pose) of a small sensor coil from eight measurements induced by a planar array of emitter coils. This is an inverse problem: the physics maps from pose to measurement, but the system must run this map backwards, recovering five pose coordinates from eight measured values.

This chapter builds the machinery needed to both understand and solve the problem. We first describe the physical system, then derive the forward map from the underlying physics. We then ask when this map can be inverted, at least locally, and turn to methods that can do this in practice.

The first class of methods we look at is the iterative methods, whose success depends on where they start. The second class are neural networks (NNs), which learn an approximate inverse directly. The chapter closes by introducing a hybrid approach, which combines the strengths of both.

1.1 The Anser System

Anser EMT [1] is the first fully open-source electromagnetic tracking (EMT) platform for image-guided interventions. It uses magnetic fields to determine the position and orientation of a sensor embedded in the field. It has two main components: a planar field generator, and a tracking sensor coil.

1.1.1 The Field Generator

The Anser EMT system uses a planar field generator consisting of 8 coils being driven at different frequencies. Each coil consists of a wound rectangular spiraling filament with specifications shown in Table 1.1.

Eight of these coils are positioned on the PCB, as seen in Figure **??**.

Property	Value
Outer side length	70 mm
Turns per coil	25
Track width	0.5 mm
Track spacing	0.25 mm
PCB thickness	1.6 mm

Table 1.1: Specification of PCB emitter coils in Anser field generator [1]

[MISSING FIGURE]
FIGURE OF SINGLE COIL

Figure 1.1: FIGURE OF SINGLE COIL

[MISSING FIGURE]
Figure of 8 coil generator

Figure 1.2: Figure of 8 coil generator

1.1.2 The Tracking Sensor Coil

The tracking sensor records a measurement related to the flux, and due to the emitter coils running at different frequencies, is able to recover the 8 measurements due to each of the emitter coils after demodulation. These 8 measurements are fed into the pose solving algorithm to determine the position and orientation of the sensor.

1.2 Magnetic Field Derivation

Each spiral coil is composed of a sequence of straight line filaments and Maxwell's equations in free space are linear [2, p. 59] so the total magnetic field will be the sum of the fields of each individual straight line filament. Our main goal then, is to find the magnetic field generated at an observer point p due to the current flowing along one of these straight line filaments, $\mathbf{a}$.

We start by defining vectors $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$.

- $\mathbf{a}$ points along the current filament.
- $\mathbf{b}$ points from the end of the filament to the observer.
- $\mathbf{c}$ points from the start of the filament to the observer.

Note that throughout we use the convention $|\mathbf{a}| = a$.

Now, we use the Biot Savart Law to find the magnetic field in each of the lines.

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \int \frac{d\mathbf{l} \times \mathbf{r}}{r^3} \tag{1.1}$$

To evaluate this integral we first parametrise $\mathbf{r}$ in a parameter t, yielding:

$$\begin{aligned} \mathbf{r}(t) &= \mathbf{c} - \mathbf{a}t,\ t \in [0, 1] \\ d\mathbf{l} &= \mathbf{a}dt \end{aligned}$$

Substituting in gives:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \int_0^1 \frac{(\mathbf{a}dt) \times (\mathbf{c} - \mathbf{a}t)}{|\mathbf{c} - t\mathbf{a}|^3} \tag{1.2}$$

Noting that $\mathbf{a} \times \mathbf{a} = 0$, and that $\mathbf{a}$ and $\mathbf{c}$ are constants, this simplifies to:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} (\mathbf{a} \times \mathbf{c}) \int_0^1 \frac{dt}{|\mathbf{c} - t\mathbf{a}|^3} \tag{1.3}$$

The denominator must now be simplified.

$$|\mathbf{c} - t\mathbf{a}|^3 = \left(c^2 + t^2a^2 - 2t(\mathbf{a}\cdot\mathbf{c})\right)^{3/2} \tag{1.4}$$

For simplicity, let $\beta = -(\mathbf{a}\cdot\mathbf{c})$.

$$|\mathbf{c} - t\mathbf{a}|^3 = \left(a^2t^2 + 2\beta t + c^2\right)^{3/2} \tag{1.5}$$

Completing the square:

$$|\mathbf{c} - t\mathbf{a}|^3 = a^3\left(\left(t + \frac{\beta}{a^2}\right)^2 - \frac{\beta^2}{a^4} + \frac{c^2}{a^2}\right)^{3/2} \tag{1.6}$$

Now let $u = t + \frac{\beta}{a^2}$, $du = dt$, $u(0) = \frac{\beta}{a^2}$, $u(1) = 1 + \frac{\beta}{a^2}$. Also, let $\delta = \frac{c^2}{a^2} - \frac{\beta^2}{a^4}$. Substituting back into our Biot Savart Formula gives:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi}\frac{\mathbf{a}\times\mathbf{c}}{a^3}\int_{\frac{\beta}{a^2}}^{1+\frac{\beta}{a^2}} \frac{du}{\left(u^2+\delta\right)^{3/2}} \tag{1.7}$$

We want to use the identity $\sec^2\psi = 1 + \tan^2\psi$, to do so, factor out $\delta^{3/2}$

$$\mathbf{B} = \frac{\mu_0 I}{4\pi}\frac{\mathbf{a}\times\mathbf{c}}{a^3\delta^{3/2}}\int_{\frac{\beta}{a^2}}^{1+\frac{\beta}{a^2}} \frac{du}{\left(\frac{u^2}{\delta}+1\right)^{3/2}} \tag{1.8}$$

Now that it's in the correct form let: $\tan^2\psi = \frac{u^2}{\delta}$, then:

$$\begin{aligned}
\tan\psi &= \frac{u}{\sqrt{\delta}} \\
\sec^2\psi\, d\psi &= \frac{1}{\sqrt{\delta}}du \\
\psi\left(\frac{\beta}{a^2}\right) &= \arctan\left(\frac{\beta}{a^2\sqrt{\delta}}\right) \\
\phi\left(1+\frac{\beta}{a^2}\right) &= \arctan\left(\frac{1+\frac{\beta}{a^2}}{\sqrt{\delta}}\right)
\end{aligned}$$

Substituting back in:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi}\frac{\mathbf{a}\times\mathbf{c}}{a^3\delta^{3/2}}\int_{\arctan\left(\frac{\beta}{a^2\sqrt{\delta}}\right)}^{\arctan\left(\frac{1}{\sqrt{\delta}}\left(1+\frac{\beta}{a^2}\right)\right)} \frac{\sqrt{\delta}\sec^2\psi\, d\psi}{\left(1+\tan^2\psi\right)^{3/2}} \tag{1.9}$$

Using the identities $\sec^2\psi = 1 + \tan^2\psi$ and $\cos\psi = \frac{1}{\sec\psi}$ and simplifying yields:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{a^3 \delta} \int_{\arctan\left(\frac{\beta}{a^2\sqrt{\delta}}\right)}^{\arctan\left(\frac{1}{\sqrt{\delta}}\left(1+\frac{\beta}{a^2}\right)\right)} \cos\psi \, d\psi \tag{1.10}$$

Solving:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{a^3 \delta} \left(\sin\left(\arctan\left(\frac{1}{\sqrt{\delta}} \left(1 + \frac{\beta}{a^2} \right) \right) \right) - \sin\left(\arctan\left(\frac{\beta}{a^2\sqrt{\delta}} \right) \right) \right) \tag{1.11}$$

Now, we must consider a right angled triangle to work this out. $\sin\psi = \frac{o}{h}$, $\tan\psi = \frac{o}{a}$ therefore $\sin\left(\arctan\left(\frac{o}{a}\right)\right) = \frac{o}{\sqrt{o^2+a^2}}$ by Pythagoras' Theorem.

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{a^3 \delta} \left(\left(\frac{1 + \frac{\beta}{a^2}}{\sqrt{\delta + \left(1 + \frac{\beta}{a^2}\right)^2}} \right) - \left(\frac{\beta}{\sqrt{\beta^2 + a^4\delta}} \right) \right) \tag{1.12}$$

Recalling $\delta = \frac{c^2}{a^2} - \frac{\beta^2}{a^4}$, $\beta = -\mathbf{a}\cdot\mathbf{c}$, and looking at each of the denominators:

$$\begin{aligned}
\sqrt{\delta + \left(1 + \frac{\beta}{a^2}\right)^2} &= \sqrt{\frac{c^2}{a^2} - \frac{\beta^2}{a^4} + 1 + 2\frac{\beta}{a^2} + \frac{\beta^2}{a^4}} \\
&= \sqrt{1 + \frac{c^2}{a^2} - \frac{2}{a^2}(\mathbf{a}\cdot\mathbf{c})} \\
&= \frac{1}{a}\sqrt{a^2 + c^2 - 2(\mathbf{a}\cdot\mathbf{c})} \\
&= \frac{1}{a}|\mathbf{a} - \mathbf{c}| \\
&= \frac{b}{a}
\end{aligned}$$

$$\begin{aligned}
\sqrt{\beta^2 + a^4\delta} &= \sqrt{\beta^2 + a^2c^2 - \beta^2} \\
&= \sqrt{a^2c^2} \\
&= ac
\end{aligned}$$

Substituting into our formula for $\mathbf{B}$ now gives:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{ac^2 - \frac{(\mathbf{a}\cdot\mathbf{c})^2}{a}} \left(\frac{1 - \frac{\mathbf{a}\cdot\mathbf{c}}{a^2}}{\frac{b}{a}} + \frac{\mathbf{a}\cdot\mathbf{c}}{ac} \right) \tag{1.13}$$

Multiplying the numerator and denominator across by $\mathbf{a}$

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{a^2 c^2 - (\mathbf{a} \cdot \mathbf{c})^2} \left(\frac{a^2 - \mathbf{a} \cdot \mathbf{c}}{b} + \frac{\mathbf{a} \cdot \mathbf{c}}{c} \right) \tag{1.14}$$

Now, picking out these two parts and simplifying, where α is the angle between $\mathbf{a}$ and $\mathbf{c}$:

$$\begin{aligned} a^2 c^2 - (\mathbf{a} \cdot \mathbf{c})^2 &= a^2 c^2 - a^2 c^2 \cos^2(\alpha) \\ &= a^2 c^2 (1 - \cos^2 \alpha) \\ &= a^2 c^2 \sin^2 \alpha \\ &= |\mathbf{a} \times \mathbf{c}|^2 \end{aligned}$$

$$\begin{aligned} a^2 - \mathbf{a} \cdot \mathbf{c} &= \mathbf{a} \cdot \mathbf{a} - \mathbf{a} \cdot \mathbf{c} \\ &= \mathbf{a} \cdot (\mathbf{a} - \mathbf{c}) \\ &= \mathbf{a} \cdot (-\mathbf{b}) \\ &= -\mathbf{a} \cdot \mathbf{b} \end{aligned}$$

This gives us our final form:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{|\mathbf{a} \times \mathbf{c}|^2} \left(\frac{\mathbf{a} \cdot \mathbf{c}}{c} - \frac{\mathbf{a} \cdot \mathbf{b}}{b} \right) \tag{1.15}$$

Now, remembering that this equation is linear and that each coil is made up of straight line filaments we get:

$$\mathbf{B}_{\text{total}} = \sum_{\text{coils}} \sum_{\text{filaments}} \mathbf{B}_{\text{filament}} \tag{1.16}$$

Now, since we are in free space we use the identity $\mathbf{H} = \frac{\mathbf{B}}{\mu_0}$ [2, p. 285]

1.3 Understanding the forward map

The problem is framed as an inverse problem. The forward map takes position and orientation data and calculates 8 measurements from each of the coils. Our goal is to approximate an inverse to this map. To do so, we first try to understand the forward map

1.3.1 Definition and properties

First we define a pose vector $\mathbf{p} \in \mathbb{R}^5$

$$\mathbf{p} = \begin{bmatrix} x \\ y \\ z \\ \theta \\ \phi \end{bmatrix} \tag{1.17}$$

In practice, $\mathbf{p}$ is restricted to a set $\Omega \subset \mathbb{R}^5$

$$\Omega = \left\{ \begin{bmatrix} x \\ y \\ z \\ \theta \\ \phi \end{bmatrix} \in \mathbb{R}^5 : \begin{array}{c} x \in (-0.125 \text{ m}, 0.125 \text{ m}) \\ y \in (-0.125 \text{ m}, 0.125 \text{ m}) \\ z \in (0.001 \text{ m}, 0.25 \text{ m}) \\ \theta \in (0 \text{ rad}, \pi \text{ rad}) \\ \phi \in [0 \text{ rad}, 2\pi \text{ rad}) \end{array} \right\} \tag{1.18}$$

Then we define a vector of measurements, $\mathbf{m} \in \mathbb{R}^8$

$$\mathbf{m} = \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ m_5 \\ m_6 \\ m_7 \\ m_8 \end{bmatrix} \tag{1.19}$$

Now we can write the forward map: $\mathbf{F}(\mathbf{p}) : \Omega \subset \mathbb{R}^5 \to \mathbb{R}^8$. More precisely,

$$m_i = \mathbf{H}_i(x, y, z) \cdot \hat{n}(\theta, \phi) \tag{1.20}$$

where $\mathbf{H}_i$ is the magnetic field calculated at (x, y, z) due to coil i using equation (1.15), and $\hat{n}(\theta, \phi)$ is the vector normal to the sensor, defined by:

$$\hat{n}(\theta, \phi) = \begin{bmatrix} \sin\theta\cos\phi \\ \sin\theta\sin\phi \\ \cos\theta \end{bmatrix} \tag{1.21}$$

Since F is smooth, the image of the 5-dimensional domain, Ω, has measure zero in $\mathbb{R}^8$, therefore F cannot be surjective.

1.3.2 The Jacobian

Now that we have established that F is not surjective we must move our attention to thinking about local invertibility rather than global. To do so we make a local approximation:

$$F(\mathbf{p}+\delta) = F(\mathbf{p}) + J(\mathbf{p})\delta \tag{1.22}$$

where **delta** is a small step in $\mathbb{R}^5$, and J is the *Jacobian*, defined by: Jacobian:

Definition 1.3.1 (Jacobian)

$$J(\mathbf{p}) = \frac{\partial F}{\partial \mathbf{p}} \in \mathbb{R}^{8\times 5},\ J_{i,j} = \frac{\partial F_i}{\partial p_j}$$

H is a smooth map away from the current filaments. $\hat{n}$ is also a smooth map, therefore F will also be smooth away from the current filaments and the Jacobian will be well defined there.

A closed form for $\mathbf{J}$ is cumbersome to find so we instead use finite differences to approximate $\mathbf{J}$

$$\mathbf{J}_{:,\mathbf{i}} \approx \frac{F\left(\mathbf{p}+h\hat{e_i}\right) - F(\mathbf{p})}{h} \tag{1.23}$$

where $\hat{e}_i$ is the unit vector in the ith direction, and h is a small real value.

1.3.3 Local Invertibility

The forward map sends $\Omega \subset \mathbb{R}^5$ into $\mathbb{R}^8$, so it is never globally invertible. This raises the question: when, if ever, is the forward map locally invertible?

The derivative of F at a point $\mathbf{p}_0 \in \Omega$ is the linear map $J_0 = J(\mathbf{p}_0) \in \mathbb{R}^{8\times 5}$. The following conditions are equivalent:

$J_0^\top J_0$ invertible $\iff$ J_0 has full column rank $\iff$ J_0 is injective as a linear map.

It can be shown [3] that if J_0 has full column rank, then F is injective on a neighbourhood of $\mathbf{p}_0$. This means that the pose is unique in that neighbourhood: distinct nearby poses produce distinct measurements, and the pose is locally recoverable from the measurement. As we will see later, in Section 1.4.1, $J^\top J$ is exactly the matrix the Gauss-Newton algorithm must invert.

Rank deficiency therefore signals regions where the pose cannot be recovered from the measurement, even locally. Full column rank is a condition for the inverse problem to be well posed.

1.4 Iterative Methods

Given a function $f : \mathbb{R}^N \to \mathbb{R}^M$ with $M \geq N$, and a given $y \in \mathbb{R}^M$ suppose we want to find an $x \in \mathbb{R}^N$ that satisfies $y = f(x)$. In the case $m = n$ we could get bijectivity which would allow us to simply invert the map, however, in our case we have $n = 5$ and $m = 8$ so we cannot. This leads us to iterative methods.

1.4.1 Gauss-Newton

The Gauss-Newton Algorithm [?] solves this exact kind of problem.

We have a function $f : \mathbb{R}^n \to \mathbb{R}^m$ and we know $y \in \mathbb{R}^m$, we want to find $x_{\text{true}} \in \mathbb{R}^n$ such that $y = f(x_{\text{true}})$. We start with an initial guess $x_0 \in \mathbb{R}^n$ and define a residual:

$$r(x) = f(x) - y \tag{1.24}$$

Then we define a cost function:

$$C(x) = \frac{1}{2} \|r(x)\|^2 = \frac{1}{2} r(x)^\top r(x) \tag{1.25}$$

Suppose now we take a step, $\delta \in \mathbb{R}^N$, we want to find the δ which minimises the cost function.

$$C(x + \delta) = \frac{1}{2} \|r(x + \delta)\|^2 \tag{1.26}$$

Now, linearising:

$$C(x + \delta) \approx \frac{1}{2} \|r(x) + J(x)\delta\|^2 \tag{1.27}$$

Since this is a scalar quantity we can take $\nabla_\delta C$ and set it to zero to find the minimum.

$$\nabla_\delta C(x + \delta) = \left(\frac{\partial}{\partial \delta} C(x + \delta) \right)^\top = 0 \tag{1.28}$$

Now, we consider the kth component of this gradient:

$$\left(\nabla_\delta C(x+\delta)\right)_k \approx \frac{1}{2}\frac{\partial}{\partial \delta_k}\sum_{j=1}^{m}\left(r_j(x)+\sum_{i=1}^{n}J_{ji}(x)\delta_i\right)^2 \tag{1.29}$$

$$\approx \frac{1}{2}\sum_{j=1}^{m}2\left(r_j(x)+\sum_{i=1}^{n}J_{ji}(x)\delta_i\right)\frac{\partial}{\partial \delta_k}\left(r_j(x)+\sum_{i=1}^{n}J_{ji}(x)\delta_i\right) \tag{1.30}$$

$$\approx \sum_{j=1}^{m}J_{jk}(x)\left(r_j(x)+\sum_{i=1}^{n}J_{ji}(x)\delta_i\right) \tag{1.31}$$

$$\approx \sum_{j=1}^{m}J_{jk}r_j+\sum_{j=1}^{m}\sum_{i=1}^{n}J_{jk}J_{ji}\delta_i \tag{1.32}$$

$$\approx \left(J^\top r\right)_k+\left(J^\top J\delta\right)_k \tag{1.33}$$

Since this applies for all k we get:

$$\nabla_\delta C(x+\delta)\approx J^\top r+J^\top J\delta \tag{1.34}$$

Setting this equal to zero and solving, yields:

$$\delta=-\left(J^\top J\right)^{-1}J^\top r \tag{1.35}$$

Now that we know the theoretically "best" step to take, we iterate and do this repeatedly:

$$x_{i+1}=x_i-\left(J(x_i)^\top J(x_i)\right)^{-1}J(x_i)^\top r(x_i) \tag{1.36}$$

This method has a known failure mode: it has no step-size control. Far from the solution, where the linearisation is poor, the full step can increase the cost and the iteration can diverge.

1.4.2 Gradient Methods

Gauss-Newton fails when it "trusts" the linear model too far from the point at which it was linearised. The opposite approach makes no model of the residual at all. It just moves in the direction that decreases the cost the fastest. Consider the cost function (1.25) and take its gradient.

$$\nabla_x C(x)=\nabla_x\frac{1}{2}r(x)^\top r(x) \tag{1.37}$$

Using the same argument as before, taking the kth component of this gradient:

$$\begin{aligned}
\left(\nabla_x C(x)\right)_k &= \frac{1}{2}\frac{\partial}{\partial x_k}\sum_{j=1}^{m} r_j(x) r_j(x) && (1.38)\\
&= \frac{1}{2}\sum_{j=1}^{m}\frac{\partial}{\partial x_k} r_j(x)^2 && (1.39)\\
&= \frac{1}{2}\sum_{j=1}^{m} 2 r_j(x)\frac{\partial r_j(x)}{\partial x_k} && (1.40)\\
&= \sum_{j=1}^{m} r_j(x)\frac{\partial (f_j(x) - y_j)}{\partial x_k} && (1.41)\\
&= \sum_{j=1}^{m} r_j(x)\frac{\partial f_j(x)}{\partial x_k} && (1.42)\\
&= \sum_{j=1}^{m} r_j(x) J_{jk}(x) && (1.43)\\
&= \left(J^\top(x) r(x)\right)_k && (1.44)
\end{aligned}$$

Gradient descent then steps against this gradient, with the step-size being denoted by the parameter $\eta \in \mathbb{R}$:

$$x_{i+1} = x_i - \eta J(x_i)^\top r(x_i) \tag{1.45}$$

Because $-\nabla_x C$ is in the direction of steepest local decrease, a sufficiently small η is guaranteed to reduce the cost, so the method is robust far from the solution where Gauss-Newton diverges. Its failure mode is also the opposite of Gauss-Newton's. Near the solution, the gradient becomes small, and since the algorithm has no knowledge of the distance to the solution it bounces back and forth around the minimum.

1.4.3 Levenberg's Contribution

In 1944, Levenberg [4] interpolated between these two algorithms. If we denote each step as δ_{GN} and δ_{GD}, for Gauss-Newton and gradient descent, respectively we get that each of these steps solves the equations:

$$\begin{aligned}
(J^\top J)\delta_{GN} &= -J^\top r && (1.46)\\
\delta_{GD} &= -\eta J^\top r && (1.47)
\end{aligned}$$

Now, letting $\lambda = \frac{1}{\eta}$, multiplying by $I \in \mathbb{R}^{n \times n}$, and interpolating between these two yields:

$$\left(J^\top J + \lambda I\right) \delta = -J^\top r \tag{1.48}$$

Here, λ is known as the damping factor, and another key insight Levenberg had was to vary λ. Since GN works better the closer to the solution we get and GD works better the further from the solution we are, the damping is varied dynamically.
We start at step i with $\lambda = \lambda_i$. If $C(x_{i+1}) < C(x_i)$ we are moving in the right direction so can move more into the GN regime, setting $\lambda_{i+1} = 0.1\lambda_i$. However, if $C(x_{i+1}) \geq C(x_i)$ then it is clear that GN is not working and we need to move into a GD dominated regime, thus setting $\lambda_{i+1} = 10\lambda_i$. This ensures that when we are far from the solution we are in a GD dominated regime and as we get closer to the solution GN starts to take over.

1.4.4 The Levenberg Marquardt Algorithm

One downfall of Levenberg's approach is that λ is the same in every dimension, even though the dimensions may operate on different scales. Marquardt [5] solves this by replacing the identity matrix, I with $D = \text{Diag}\left(J^\top J\right)$ to scale the weights, where Diag is the operator which acts on matrices by setting the off-diagonal elements to zero and operates on vectors by making a diagonal matrix out of its components. The Levenberg-Marquardt algorithm solves the following equation at every iteration:

$$\left(J^\top J + \lambda D\right) \delta = -J^\top r \tag{1.49}$$

This means that the algorithm works with different damping in each direction, proportional to the "scale" of that particular direction.

1.4.5 Convergence and the Role of Initialisation

Both Gauss-Newton and Levenberg-Marquardt are local methods: they converge to a stationary point of C, and which stationary point depends on where they start. Since C is typically non-convex it can have multiple local minima. In our problem, in particular, given the high levels of symmetry between the coils, it is highly likely that there are multiple minima. These iterative methods work when the initial guess is in the correct basin of attraction. That then raises the question, how do we get into the right basin of attraction?

[MISSING FIGURE]
figure of local minima

Figure 1.3: figure of local minima

1.5 Neural Networks for learning inverses

The iterative methods above invert the forward map each time for every measurement. An alternative would be to spend the computation time upfront and approximate a full inverse map once.

1.5.1 Feed Forward Neural Network

We use a fully-connected feed forward neural network (FFNN) to approximate the inverse. Suppose we have a function $\Phi : \mathbb{R}^N \to \mathbb{R}^M$ an approximate inverse is defined on the image of Φ, which we call f. We would like to find $y \in \mathbb{R}^N$ such that $y = f(x), x \in \mathbb{R}^M$. To do so, a FFNN with L hidden layers performs the following composition, with $o_0 = x$ and $o_L = \hat{y}$:

$$o_{i+1} = \sigma\left(W_i o_i + b_i\right) \tag{1.50}$$

where W_i and b_i are the weight matrices and bias vectors (collectively the parameters w), and σ is the nonlinear activation function.
We represent this approximation as $\hat{f}_w : \mathbb{R}^M \to \mathbb{R}^N$, with $\hat{y} = \hat{f}_w(x)$

The question then becomes how do we find the collection of parameters, w, which does this?

1.5.2 Training a FFNN

Because we possess F we can generate unlimited training data, a collection of labelled pairs, (x_i, y_i). The goal of training is then to minimise the *loss function* with respect to the parameters, w:

Definition 1.5.1 (Loss Function)

$$\hat{L}(w) = \frac{1}{N} \sum_{i=1}^{N} \left\| \hat{f}_w(x_i) - y_i \right\|^2$$

We minimise the loss using an optimiser, in our case the Adam optimiser [6]. Generalisation is then assessed on a held-out test set never seen during training.

Chapter 2

Methods

The previous chapter ended with a question. Levenberg-Marquardt converges to a stationary point of the cost function, and which stationary point it reaches is determined by where it begins. The solver is only as good as its initial guess. This chapter builds a neural-network based initialiser which then provides an initial guess for LM to allow it to converge to the correct stationary point.

Section 2.1 implements the forward map in Python and fixes the sampling scheme. Section 2.4 characterises the Levenberg-Marquardt solver, and shows implementation considerations. Section 2.2 establishes the importance of choosing a good method to quantify "closeness", particularly with respect to angle. Section 2.5 determines the limits of a neural-network based solver and motivates the need for LM refinement. Section 2.6 joins the two halves.

2.1 Data generation

2.1.1 Anser Simulation in Python

We simulate the Anser system in Python, developing on a MATLAB simulation by Jaeger et al. [1]. [1] The pipeline proceeds as follows:

Coil geometry First we generate the vertex coordinates for a square spiral PCB emitter coil. This is done in a local coordinate system and then copies are rotated and translated to place them in the global coordinate system.

[1]The implementation is available at `https://github.com/pmccarthy3901/ansermodelling`. A mapping from the sections of this chapter to the corresponding modules is given in Appendix **??**.

Each coil is a square which spirals in, it has N turns per layer, the side-length of the outermost square is l, the trace width is w, there is a line to line spacing of s and the thickness of the PCB is z_{thick}. After the coil has done N_{turns} turns on the upper layer, it is connected with a via to the lower layer where the coil spirals out again.

Conversion from Local to Global Coordinates As all of the coils are the same shape, only one coil is generated, then it is copied a number of times into different places in space. The module selects an axis about which the rotation takes place. This is represented as an integer $(0, 1, 2) - (x, y, z)$. This works nicely because the rotation matrix can be generated using an even permutation of the axes. The module then applies the permuted rotation matrix to a copy of the input vector and then applies a translation.

Field Calculation The calculation of the magnetic field generated by the coils is done in `field_coil_calc`. It uses the linearity of the Biot-Savart line integral to calculate the total magnetic field due to the coils at an observer point as the sum of the contributions of each straight-line segment. It then implements equation (1.15) to calculate the magnetic field.

Objective Function The objective function takes in the real observed measurement, and returns the difference between it and the measurement calculated using the simulation. It calculates the simulated measurement as:

$$\mathbf{m} = \mathbf{H}_{\text{sim}} \cdot \hat{n}_{\text{obs}} \tag{2.1}$$

where:

- $\mathbf{H}_{\text{sim}}$ is the simulated magnetic field
- $\hat{n}_{\text{obs}}$ is the unit vector along the sensor axis.

2.1.2 Sampling

When generating data, it is important that our sampling space is uniform. For position, this is easy, we sample uniformly in the bounds of the box. The angle, however requires a bit more consideration.

2.1.2.1 Sampling the position

The anser system operates in a 25x25x25cm box.

Variable	Min (mm)	Max (mm)
x	-125	125
y	-125	125
z	1.6	250

Table 2.1: Bounds of the sampling volume.

Variable	Min	Max
$\cos\theta$	-1	1
ϕ	0 rad	2π rad

Table 2.2: Bounds of the orientation sampling volume.

Since the sensor coil is on a PCB that is 1.6mm thick, the z value has a minimum of $z = 0.0016$m. Therefore we sample uniformly within the bounds shown in Table 2.1

2.1.2.2 Sampling the orientation

Consider an area element dA on the unit sphere, seen in Figure **??**. Then the area element is $dA = \sin\theta d\theta d\phi$. Therefore, to uniformly sample on the sphere we must not sample uniformly in θ, but rather in $\cos\theta$. The orientation sampling is shown in Table 2.2.

2.1.3 Dataset size, split, and normalisation

The dataset consists of $N = 100000$ pose-measurement pairs. The measurements are the model input and the pose is the regression target. The data is split 80/20 into training and validation sets, and the validation set is never seen during training. Because the measurement magnitudes vary over several

[MISSING FIGURE]

FIGURE OF AREA ELEMENT ON UNIT SPHERE

Figure 2.1: FIGURE OF AREA ELEMENT ON UNIT SPHERE

orders of magnitude across the volume, the inputs are normalised using the mean and standard deviation of the training set. The same statistics are applied to the validation set to prevent data leakage.

2.2 Representing the pose

The pose contains two angles, and how they are represented as regression targets matters more than it first appears. Regressing against θ and ϕ directly has three problems. First, ϕ is periodic: $\phi = 0.01$ and $\phi = 2\pi - 0.01$ are only separated by 0.02 rad, however a naive loss would heavily punish such a situation. The second problem is with ϕ: at the poles $\theta \in \{0, \pi\}$, ϕ is not defined. The third problem is more general: squared error in the angles is not a distance on the sphere. The metric induced on the unit sphere, S^2, by the parametrisation carries an extra factor of $\sin^2\theta$:

$$ds^2 = d\theta^2 + \sin^2\theta \, d\phi^2 \tag{2.2}$$

For these reasons, it becomes natural to represent the orientation not in terms of angles, but in terms of the unit normal itself:

$$\hat{n}(\theta, \phi) = \begin{bmatrix} \sin\theta\cos\phi \\ \sin\theta\sin\phi \\ \cos\theta \end{bmatrix} \tag{2.3}$$

The normal vector is smooth, unique, and has no wrap-around. We therefore train networks whose target is the vector:

$$t = (x, y, z, n_1, n_2, n_3) \tag{2.4}$$

where n_i is the unit normal in the ith direction.

This argument, however, does not apply to the Levenberg-Marquardt solver of Section 2.4, which continues to operate in the (θ, ϕ) regime. This is because an iterative solver initialised near its solution doesn't encounter these problems and, conversely, if we were to regress against the unit normal we would have the issue of having 6 parameters for 5 degrees of freedom meaning that $J^\top J$ wouldn't be invertible, contradicting the exact condition that LM needs.

2.3 Error metrics

In order to quantify how "good" a particular model is, we need to define our errors. Since the pose has two components: position and orientation, we define an error on each.

2.3.1 Positional Error

The positional error e_p is defined using the euclidean distance between the true position, x_{true}, and the predicted position, x_{pred}.

$$e_p := \|x_{\text{true}} - x_{\text{pred}}\|_2 \tag{2.5}$$

2.3.2 Angular Error

The angular error, $e_{\hat{n}}$, is defined in relation to the unit normal orientations, where $\hat{n}_{\text{true}}$ is the true unit normal, and $\hat{n}_{\text{pred}}$ is the predicted unit normal:

$$e_{\hat{n}} := \arccos\left(\hat{n}_{\text{true}} \cdot \hat{n}_{\text{pred}}\right) \tag{2.6}$$

2.4 The Levenberg-Marquardt solver

The previous chapter derived the Levenberg-Marquardt update function (1.49). This section describes the implementation and parameter choice.

2.4.1 Implementation

The solver is implemented in NumPy (`models/LM_solve.py`). Its components are as follows:

Jacobian. As discussed in the previous chapter, we approximate the Jacobian using finite difference with $h = 10^{-5}$ (1.23)

Scaling. Following Marquardt's implementation [5], the damping matrix is given by:

$$D = \text{diag}(\max(\text{diag}(J^\top J), 10^{-12})) \tag{2.7}$$

where the floor of 10^{-12} guards against a zero diagonal entry making D singular and diag is the function defined in Section 1.4.3

Constraint Handling. At each iteration, constraints are applied to both the position and orientation. The position is clipped to within the box shown in Table 2.1. The orientations are handled by folding θ through the pole with a π shift in ϕ, and wrapping ϕ. This keeps the orientation unchanged on the unit sphere and alters only its coordinates.

Damping. Damping is done as in Section 1.4.3. λ_0 is set to $\lambda_0 = 10^{-5}$ and $\lambda_{\max}$ is set to $\lambda_{\max} = 10^7$.

Stopping. Iteration terminates when any of three conditions are met:

1. the l_∞ norm of the gradient falls below a threshold, ϵ_{grad}.
2. The damping parameter, λ exceeds $\lambda_{\max}$, indicating that no downhill step can be found.
3. A maximum number of iterations is reached, indicating that the solver cannot find a solution.

Only the first of those three stopping criteria indicates success, and even then, the solver successfully reaching a minimum doesn't mean that it's successfully reached the correct minimum. Therefore we have to define convergence.

We say that a solution has converged if for some ϵ_p and some ϵ_n "acceptable errors":

$$e_p < \epsilon_p$$
$$e_{\hat{n}} < \epsilon_n$$

2.5 Neural Network Solver

Section 2.4 establishes that the LM solver needs a good initial guess to converge. This section builds a neural network solver, the main advantage of which is its ability to learn the global configuration of the system and thus work on a "cold start". This section shows the choice of architecture, loss function, and training methods.

2.5.1 Neural Network Architecture

The neural network solver uses a basic Feed Forward Neural Network (FFNN) which takes the 8 input measurements and outputs the 6 dimensional pose (x, y, z, n_1, n_2, n_3). The neural network architecture is shown in Table 2.3

2.5.2 Choice of loss function

In Section 2.3 we defined two errors: e_p, the positional error, and $e_{\hat{n}}$, the angular error. Now we must define a loss function. The only problem is that these errors operate on completely different scales. Given the bounds of our box, e_p varies between 0 and $w_p = \sqrt{0.25^2 + 0.25^2 + 0.25^2} \approx 0.433$ and $e_{\hat{n}}$

Property	Value
Input dimension	8 (measurements)
Output dimension	6 (x, y, z, n_1, n_2, n_3)
Hidden layers	3
Units per layer	(256,256,256)
Activation	tanh

Table 2.3: Network architecture.

Parameter	Value
Optimiser	Adam
Learning rate	10^{-3}
Schedule	Cosine Annealing
Batch size	32
Epochs	200

Table 2.4: Training configuration.

varies between 0 and $w_{\hat{n}} = \pi$. To give equal weight to both components we define our loss, $\mathcal{L}$, as the weighted sum between squares of the two as follows:

$$\mathcal{L} = e_p^2 + \frac{w_p^2}{w_{\hat{n}}^2} e_{\hat{n}}^2 \tag{2.8}$$

This weight is chosen such that the two terms have equal range over the sampling volume.

2.5.3 Neural Network Training

The network is trained with the Adam optimiser [**?**] to minimise (2.8) over the dataset described in Section 2.1, using the settings and hyperparameters described in Table 2.4

The learning rate uses a cosine scheduler [**?**], decaying from its initial value to zero, following a cosine curve. This allows for a high learning rate at the start, which then decays smoothly at every epoch. This means that at the start the model learns quickly, and at the end the model has sufficiently small learning rate to not oscillate around the minimum.

2.6 Hybrid Model

Sections 2.4 and 2.5 present a LM based solver and a NN solver, respectively. The shortcomings of these two solvers are complementary. A hybrid model combines these two, feeding the output of the NN model into a LM solver. This allows the NN to provide a good initialisation for the LM solver to further refine.

2.6.1 Coupling the two solvers

The two components operate in different parametrisations. The NN uses (x, y, z, n_1, n_2, n_3) and the LM solver uses (x, y, z, θ, ϕ). This means that before feeding the NN output into the LM solver, we have to convert the parametrisation.

Given the predicted normal $\hat{n} = (n_1, n_2, n_3)$, normalised to unit length, the corresponding θ and ϕ are given by:

$$\begin{aligned} \theta &= \arccos(n_3) \\ \phi &= \text{atan2}(n_2, n_1) \end{aligned}$$

where atan2 is the two argument arctangent defined by:

$$\text{atan2}(y, x) = \begin{cases} \arctan\left(\frac{y}{x}\right) & \text{if } x > 0, \\ \arctan\left(\frac{y}{x}\right) + \pi & \text{if } x < 0 \text{ and } y \geq 0, \\ \arctan\left(\frac{y}{x}\right) - \pi & \text{if } x < 0 \text{ and } y < 0, \\ +\frac{\pi}{2} & \text{if } x = 0 \text{ and } y > 0, \\ -\frac{\pi}{2} & \text{if } x = 0 \text{ and } y < 0, \\ \text{undefined} & \text{if } x = 0 \text{ and } y = 0. \end{cases} \tag{2.9}$$

The use of atan2 instead of the normal arctangent allows us to both get in the correct quadrant and also mitigates against the singularity caused when $n_1 = 0$.

Bibliography

[1] Herman Alexander Jaeger, Alfred Michael Franz, Kilian O'Donoghue, Alexander Seitel, Fabian Trauzettel, Lena Maier-Hein, and Pádraig Cantillon-Murphy. Anser EMT: the first open-source electromagnetic tracking platform for image-guided interventions. *International Journal of Computer Assisted Radiology and Surgery*, 12(6):1059–1067, 2017.

[2] David J. Griffiths. *Introduction to Electrodynamics*. Pearson, Boston, 4 edition, 2013.

[3] Benjamin McKay. *Lectures on Differential Geometry*. University College Cork, Cork, Ireland, 2022. Lecture notes, University College Cork.

[4] Kenneth Levenberg. A method for the solution of certain non – linear problems in least squares. *Quarterly of Applied Mathematics*, 2:164–168, 1944.

[5] Donald W. Marquardt. An algorithm for least-squares estimation of non-linear parameters. *Journal of the Society for Industrial and Applied Mathematics*, 11(2):431–441, 1963.

[6] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.